

PRACTICE EXAM 3 OF 4 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 3

Twenty new questions. Sixty questions in, the pattern should be readable.

QUESTIONS 20 RUNNING TOTAL 60 PAUSE 9 seconds

What to watch for this time

- ◆ **Sixty questions is a full paper's worth.** Three tallies is a trend, not a coincidence.
- ◆ **Watch your pace.** If you are rushing the last five, that is the finding, not the score.
- ◆ **Notice questions you got right for the wrong reason.** A lucky guess counts on the day and not in your tally.
- ◆ **Note any question where two options felt equal.** That is a confusion pair you have not settled.

ON THE EXAM

Right for the wrong reason is the hardest thing to self-report, and the most useful.

A report-generation agent fails on large inputs. Two distinct failures appear in the logs.

In the first, the response comes back with `stop_reason max_tokens` and the output is cut off mid-sentence. In the second, the request is rejected before generation begins with an error naming the model's context window.

An engineer proposes raising `max_tokens` to fix both.

What should you tell them?

- ☐ A Raising `max_tokens` fixes both, since both are token limits
- ☐ B It fixes the first. The second needs less input: trim history, summarise, or split the work
- ☐ C Neither is fixable by configuration; both need a larger model
- ☐ D It fixes the second. The first needs a retry with backoff

◆ PAUSE AND DECIDE

What should you tell them?

- A

 Fixes both
They are different limits. One is a ceiling you chose; the other is the whole request not fitting.
- C

 Both need a bigger model
A larger window would help the second and delay it. The first is your own ceiling.
- B

 Fixes one; send less for the other
max_tokens caps output. The window covers input plus output, and no number raises it.
- D

 Reversed
Backoff is for transient failures. Truncation is deterministic.

THE SIGNAL

One is a ceiling you set. The other is a wall you cannot move. The fixes are opposites. Module 1.

A monorepo has a single CLAUDE.md of about 1,100 lines covering a Swift app, a Kotlin app, a Go service and the Helm charts.

Engineers report that a large amount of context is consumed before any work starts, and that Claude occasionally applies Swift conventions to Kotlin files.

One engineer proposes splitting it into four files imported from the root CLAUDE.md.

What is the most effective change?

- ☐ A Split into four files and import them from the root CLAUDE.md as proposed
- ☐ B Split into path-scoped rules under .claude/rules/ so each loads only when work touches its paths
- ☐ C Move the Swift and Helm sections into CLAUDE.local.md
- ☐ D Add a line instructing Claude to apply only the relevant section

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Split and import
Tidier to read, identical to load. All four imports resolve into context every session.
- B

 Path-scoped rules
Only the relevant conventions load, fixing the context cost and the cross-contamination together.
- C

 Into CLAUDE.local.md
Same volume, now gitignored, so the team loses shared conventions.
- D

 Instruct it to be selective
The content is already in context. Asking it to ignore what it can see is not context management.

THE SIGNAL

Imports always load. Path scoping is what actually reduces context. Module 8.

An insurance claim classifier assigns claims to seven categories. The categories are clearly defined and mutually exclusive, and accuracy is 93%.

The errors concentrate on genuinely borderline claims: storm damage that is also a maintenance failure, theft that is also a liability question.

The team wants to improve accuracy on those.

What is most likely to help?

- ☐ A Rewrite the seven category definitions to be more precise
- ☐ B Add few-shot examples of borderline claims with their correct categories and brief reasoning
- ☐ C Add an eighth category called "mixed"
- ☐ D Instruct the model to be more careful with claims that could fit two categories

◆ PAUSE AND DECIDE

What is most likely to help?

- A

 More precise definitions
The definitions are already clear. The difficulty is which of two clear categories applies.
- B

 Borderline examples with reasoning
The boundary is a judgement call, and judgement is easier to demonstrate than to define.
- C

 A "mixed" category
Moves hard cases into a pile nobody has a process for.
- D

 Be more careful
An adjective. It does not tell the model how to resolve a tie it can already see.

THE SIGNAL

Boundary failure, not scope failure. Examples for what is easier to show than to define. Module 12.

An engineer needs the agent to find every file whose name matches `migration_*.sql`, and separately every file that references a table called `legacy_orders`.

The repository has about 3,000 files.

Which tools, in which order?

- ☐ A Grep for the filename pattern, then Glob for the table reference
- ☐ B Glob for the filename pattern, then Grep for the table reference
- ☐ C Grep for both, since Grep can match filenames in content
- ☐ D Read the directory tree, then read each SQL file

◆ PAUSE AND DECIDE

Which tools, in which order?

A Grep then Glob

The reversal. Glob does not look inside files, so it cannot find a table reference.

C Grep for both

Possible for the filenames, and Glob is purpose-built and cheaper for path matching.

B Glob then Grep

Names are paths, so Glob. Table references are content, so Grep. Neither loads a file.

D Read everything

Loads every SQL file to answer what two searches answer for nothing.

THE SIGNAL

Glob is for names. Grep is for contents. Option A is the correct answer reversed. Module 16.

A bank routes inbound secure messages to teams. A rules engine currently classifies 97% of them correctly using sender type and a small keyword set. The remaining 3% go to a triage officer.

Volume is high and the routing must happen within seconds. Costs are under scrutiny.

An architect proposes replacing the whole thing with an agent that has classification, routing and escalation tools.

What should you advise?

- ☐ A Build the agent; one system handling both common and unusual cases is simpler
- ☐ B Keep the rules engine for the 97% and send only the unclassified remainder to a model
- ☐ C Build the agent, capped with max_turns and a budget limit to control cost
- ☐ D Replace the rules engine with a single classification call, keeping the human triage step

◆ PAUSE AND DECIDE

What should you advise?

- A

 Build the agent
Puts a slow, costly decision-maker in front of 97% of traffic a free rule already handles.
- B

 Keep the rule, escalate the rest
The simplest thing that works. The model is spent only where the path is unknown.
- C

 Agent with caps
Caps limit the damage from a design that should not exist. They do not make it right.
- D

 Model classifier
Adds cost and latency to replace something already correct.

THE SIGNAL

"A rules engine already classifies them correctly" is the planted detail. Nothing is broken there. Module 1.

A reporting agent connects to a warehouse through an MCP server. Before it can build any query it calls `list_dimensions`, then calls `describe_dimension` two or three times to find the right attributes, then runs the query.

The dimension model changes about once a quarter. Simple questions take four or five tool calls.

What is the most effective change?

- ☐ A Merge `list_dimensions` and `describe_dimension` into one tool returning everything
- ☐ B Expose the dimension model as an MCP resource so it is in context before the agent plans
- ☐ C Cache the dimension model in the system prompt and update it quarterly by hand
- ☐ D Add the dimension list to the query tool's description

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Merge the two tools
Better than three calls, and still a call. It discovers at run time what it could have known.
- B

 A resource
Stable, referenced constantly, needed before deciding anything. The resource case exactly.
- C

 Paste it in the system prompt
Works until someone forgets the quarterly update. The server already knows it.
- D

 In the tool description
Descriptions are for how to use a tool. A model there goes stale and bloats every request.

THE SIGNAL

Needs to know it before deciding means resource. Needs to do it means tool. Module 7.

A team requires that Claude Code can never read files under a directory containing customer personal data. There are no exceptions and no time windows: the rule holds always.

The directory path is fixed and known.

This pairs with exam 1 question 9, where a hook was correct.

What enforces this most appropriately?

- ☐ A A PreToolUse hook on Read that inspects the path and blocks matches
- ☐ B A permissions deny rule on Read for that path
- ☐ C A CLAUDE.md rule naming the directory as off limits
- ☐ D A PostToolUse hook that alerts when the directory is read

◆ PAUSE AND DECIDE

What enforces this most appropriately?

- A

 PreToolUse hook
Genuine enforcement, and it is code you now own and must test for a rule with no condition in it.
- C

 CLAUDE.md rule
Context. It persuades and cannot guarantee.
- B

 Permissions deny
Flat rule, fixed path, no exceptions. Declarative, auditable, nothing to maintain.
- D

 PostToolUse alert
The file was already read into context. The leak has happened.

THE SIGNAL

The pair: exam 1's rule was time-conditional, so a hook. This one is flat, so a permission. Module 9.

A team generates marketing copy from product specifications. Output is grammatical and on-brand but frequently misses a required regulatory disclaimer, which is a clear pass-or-fail property.

A single generation call is used. Reviewers catch the missing disclaimer about half the time before publication.

What is the most appropriate structure?

- ☐ A A generation call followed by an evaluator call that checks the disclaimer and returns the copy for revision if missing
- ☐ B A stronger prompt requiring the disclaimer in every output
- ☐ C Three parallel generations, publishing whichever includes the disclaimer
- ☐ D A subagent that writes the disclaimer separately, appended after generation

◆ PAUSE AND DECIDE

What is the most appropriate structure?

- A

Evaluator-optimiser
A clear pass-or-fail criterion and a revisable output. That is exactly when a critic earns its keep.
- B

Stronger prompt
Improves the rate. "Required" and "pass or fail" are not rates.
- C

Three generations
Three times the cost to gamble that one includes it. Also may produce none.
- D

Append it separately
Plausible, and a disclaimer bolted on may not fit the copy or its placement rules.

THE SIGNAL

A clear evaluation criterion plus a revisable output is the evaluator-optimiser case. Module 2.

A shipping extraction schema has a dimensions field defined as a string, because documents write sizes as "120 x 80 x 60 cm", "48 x 32 x 24 in" and "1.2m x 0.8m x 0.6m".

Downstream, a service parses these strings to check container fit. It has been wrong four times this year, three of them on imperial measurements.

The team wants this to stop.

What should change?

- ☐ A Improve the downstream parser to handle more formats
- ☐ B Extract numeric length, width and height plus a separate unit enum, normalised at extraction
- ☐ C Instruct the model to convert all dimensions to centimetres
- ☐ D Reject dimension strings the parser cannot read

◆ PAUSE AND DECIDE

What should change?

- A

 Better parser
Parsing downstream, where the document context is gone. Every new format is a production bug.
- B

 Values and a unit, separated
The model reads the unit in context; your code does maths on numbers. Neither guesses.
- C

 Convert to centimetres
Better than a string, and it hides the original and makes the model do unit maths silently.
- D

 Reject unparseable
Turns silent errors into rejected records. Safer, and still not extracted.

THE SIGNAL

Normalise where the context is. The model sees "in" next to the number; the parser sees a string. Module 13.

An invoice-matching system routes anything below 0.7 confidence to human review. Reviewers correct about a quarter of what they see.

An audit of records never reviewed finds an error rate of about 5%, concentrated in invoices from one supplier whose layout the model handles confidently and incorrectly.

The team is considering raising the threshold to 0.85.

What is the most effective change?

- ☐ A Raise the threshold to 0.85 so more records are reviewed
- ☐ B Add a sampled review of records scoring above the threshold, stratified by supplier
- ☐ C Retrain reviewers to be more thorough
- ☐ D Ask the model to assign confidence more conservatively

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Raise the threshold
The errors are confident ones. A higher bar reviews more uncertain records and still misses these.
- B

 Sample above the threshold
Confident errors are only findable by looking at confident output. Stratifying targets the cluster.
- C

 Better reviewers
Those records never reach a reviewer.
- D

 More conservative scoring
It is not hesitating. It misread a layout, which does not feel uncertain.

THE SIGNAL

"Confidently and incorrectly" rules out every answer that adjusts the confidence mechanism. Module 17.

A coordinator runs a market analysis with four subagents. The final subagent writes the executive summary and consistently omits conclusions the pricing subagent produced. Logs confirm the pricing subagent completed and returned a detailed result to the coordinator. The summary subagent is spawned with the prompt "Write an executive summary of the analysis."

What is the root cause?

- ☐ A The summary subagent's turn cap is too low to gather the pricing work
- ☐ B The pricing subagent should pass its conclusions to the summary subagent directly
- ☐ C The coordinator is not including the pricing conclusions in the summary subagent's prompt
- ☐ D The summary subagent needs a more capable model

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Turn cap too low
More turns to search a context that will never contain the conclusions.
- B

 Direct subagent messaging
Invented. Subagents have no channel to each other.
- C

 Conclusions not in the prompt
The quoted prompt contains no conclusions, and the prompt is the only channel in.
- D

 Bigger model
A larger model omits data it never received just as reliably.

THE SIGNAL

When a stem quotes a prompt in full, read it as evidence. The prompt is the only channel in. Module 3.

An analytics agent has a tool `get_events` that returns every event for an account with no default limit and no date filter. Busy accounts have tens of thousands of events.

Engineers report that after two or three tool calls the agent's answers become vaguer and it forgets earlier constraints.

Most questions concern the last seven days.

What is the most effective change?

- ☐ A Move to a model with a larger context window
- ☐ B Add date range parameters and a sensible default limit, and describe when the full history is appropriate
- ☐ C Have the agent summarise the event list after receiving it
- ☐ D Truncate the response to the most recent 100 events

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Larger window
Buys headroom and changes nothing. The tool still returns everything on every call.
- B

 Date range and a default limit
Gives the agent the narrow option it lacked, and a default that is right when it does not choose.
- C

 Summarise after receiving
The full list already entered context. The summary adds tokens on top.
- D

 Silent truncation to 100
The agent believes it saw everything and answers confidently from a partial view.

THE SIGNAL

Truncating without saying so turns a cost problem into a correctness problem. Module 5.

An engineer has a skill that audits a repository for hard-coded secrets. It greps broadly, reads candidate files, and produces a twelve-line report.

Running it fills a large part of the context window with grep output and file contents. Afterwards Claude is noticeably worse at the task the engineer was working on.

What is the most effective change?

- ☐ A Add context: fork to the skill's frontmatter
- ☐ B Add allowed-tools listing only Read and Grep
- ☐ C Add disable-model-invocation so it runs less often
- ☐ D Move the skill into a plugin so it loads separately

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 context: fork
The exploration happens in an isolated window and only the twelve-line report returns.
- B

 allowed-tools
Restricts capability and trims tool definitions. The grep output is still enormous.
- C

 disable-model-invocation
Runs the same problem less often. The engineer wants to run it.
- D

 Into a plugin
Plugins are distribution. Packaging differently does not change what enters context.

THE SIGNAL

A context window of working and a twelve-line result is the fork signature. Module 9.

A payroll extraction pipeline produces records that pass schema validation with correct types on every field.

A spot check finds that in about 3% of records the stated gross pay does not equal the sum of the extracted pay components. Nobody had been checking, because validation was green.

What is the most effective change?

- ☐ A Use grammar-constrained structured outputs so the schema is strictly enforced
- ☐ B Extract both the stated gross and the components, and compare them in code, flagging mismatches
- ☐ C Add a stricter numeric type for the gross pay field
- ☐ D Ask the model to check its arithmetic before returning

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Constrained outputs
Guarantees shape, which they already have. A grammar cannot express "gross equals the sum".
- B

 Cross-check in code
A business rule needs a validation layer. The redundancy makes the error visible.
- C

 Stricter type
The type is already correct. The number is wrong, which no type expresses.
- D

 Check its arithmetic
Self-verification in the same context, and no record when it fails.

THE SIGNAL

"Passes validation with correct types" rules out every answer that strengthens the schema. Module 13.

An agent processes membership renewals. It validates the member, records the renewal, and charges the card. The charge is not idempotent.

One run failed partway: the member was validated and the renewal recorded, then a network error interrupted the run before the charge. The retry logic restarts the task from the beginning with the same input.

Duplicate renewal records are appearing.

What is the most appropriate change?

- | | |
|--|--|
| <p>☐ A Add a longer timeout so network errors interrupt runs less often</p> | <p>☐ B Resume the failed session rather than restarting, and make the side-effecting tools idempotent</p> |
| <p>☐ C Wrap the whole task in a database transaction that rolls back on failure</p> | <p>☐ D Have the agent check for an existing renewal record before creating one</p> |

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

 Longer timeout
Fewer interruptions, same behaviour when one happens.
- B

 Resume plus idempotent tools
Resuming shows the agent what it already did; idempotency covers the rest.
- C

 Wrap in a transaction
The card charge is an external side effect. No transaction rolls that back.
- D

 Agent checks first
The right idea in the wrong layer. Correctness depends on the agent remembering.

THE SIGNAL

Two options are right-shaped. D puts the check in the agent; B puts it in the tool.
Module 4.

A pricing tool calls a supplier API. When rate limited it returns HTTP 429; when a product code does not exist it returns HTTP 404. The tool reports both as `isError` with the message "supplier lookup failed".

The orchestration layer retries any failed tool call three times with backoff. Rate-limit failures usually recover. Unknown-code failures burn three retries every time.

What is the most effective change?

- ☐ A Reduce the retry count from three to one
- ☐ B Distinguish the two in the error, with a retryable flag and a message naming the cause
- ☐ C Report unknown codes as a protocol error so the retry logic skips them
- ☐ D Increase the backoff interval so retries cost less

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Fewer retries
Helps the unknown code and hurts the rate limit, which needed the retries.
- B

 Retryable flag plus cause
Two different failures deserve two different results, and each then gets the handling it needs.
- C

 Protocol error for 404
The tool exists and ran. It also hides the message from the model.
- D

 Longer backoff
Makes the wasted retries slower.

THE SIGNAL

One error shape for two failures forces one policy on both. Distinguish at the source. Module 6.

A subscriptions agent has clear cancellation policy in its system prompt. After a customer described a house move, the agent waived an early-termination fee that policy does not permit waiving.

The team adds an escalation rule: "escalate any request mentioning relocation".

Three months later the agent waives the fee for a customer who described a job loss.

What is the underlying problem?

- ☐ A The rule should have listed more circumstances, including redundancy
- ☐ B The criterion was written from an instance. It should be "any request for an exception to stated policy"
- ☐ C The system prompt needs stronger wording about the termination fee
- ☐ D The agent should not handle emotionally sensitive conversations

◆ PAUSE AND DECIDE

What is the underlying problem?

- A

 List more circumstances
The same mistake a third time. There is always another circumstance nobody listed.
- B

 Property, not instance
Both incidents share one property: an exception was requested. Neither shares the circumstance.
- C

 Stronger wording
The policy was already clear. Persuasion losing to a sympathetic case is the situation.
- D

 Exclude sensitive conversations
Over-broad, and it needs a definition of sensitive, which is the same problem again.

THE SIGNAL

Write the rule at the level the incidents share. Module 17.

A team reports an intermittent defect: under some conditions a scheduled export writes the same row twice. They describe the symptom to Claude Code and ask for a fix.

Claude makes a plausible change and reports the issue resolved. It recurs three days later. This has now happened four times with four different changes.

What is the most effective change to how they work?

- ☐ A Give Claude more detail about the conditions under which it occurs
- ☐ B Write a test that reproduces the duplicate write, and require it to fail before any fix
- ☐ C Use plan mode so the approach is reviewed before implementation
- ☐ D Ask Claude to explain why its fix works before accepting it

◆ PAUSE AND DECIDE

What is the most effective change to how they work?

- A

 More detail
Four attempts already failed on description. A fifth is the same method with more words.
- B

 A reproducing test
Turns "reported resolved" into "provably resolved", and leaves a regression guard.
- C

 Plan mode
Reviews the approach. Nobody knew the approach was wrong, which is the problem.
- D

 Explain the fix
A confident explanation of a wrong fix is what they have had four times.

THE SIGNAL

"Reports the issue resolved" is self-assessment with no external check. Module 10.

A batch of 18,000 extraction requests completes. The results file shows 17,760 succeeded and 240 errored, mostly on unusually long documents.

An engineer proposes resubmitting the entire batch after raising `max_tokens`, to keep the pipeline simple. Each record costs roughly the same to process.

What should they do instead?

- ☐ A Resubmit the whole batch as proposed; simplicity is worth the cost
- ☐ B Resubmit only the 240 failed records, matched by `custom_id`, with a higher `max_tokens`
- ☐ C Switch the pipeline to synchronous requests so errors surface immediately
- ☐ D Raise `max_tokens` for future batches and accept the loss of these 240

◆ PAUSE AND DECIDE

What should they do instead?

- A

 Resubmit everything
Reprocesses 17,760 successes to fix 240. Simplicity at seventy-four times the necessary cost.
- C

 Go synchronous
Doubles the price of the whole workload to improve error visibility on 1.3% of it.
- B

 Resubmit the failures only
Errors are per-row and custom_id matches results to records. That is what it is for.
- D

 Write off the failures
240 documents silently unprocessed.

THE SIGNAL

Per-row error reporting exists so you can resubmit selectively. Module 14.

An agent refactored four files during a session. The engineer then reverted all four on disk using version control, and resumed the session.

They expect the agent to understand the files are back to their original state and to plan accordingly.

Is that expectation correct?

- ☐ A Yes. Resuming restores the session's view of the filesystem
- ☐ B No. Sessions persist the conversation, not the files. It still believes the edits are in place
- ☐ C Yes, provided checkpointing was enabled for the session
- ☐ D No, unless the engineer runs /context before continuing

◆ PAUSE AND DECIDE

Is that expectation correct?

- A

 Resuming restores files
Conflates the two mechanisms. A session has no view of the filesystem to restore.
- B

 Conversation, not files
The conversation records the edits. Reverting the files does not revert that record.
- C

 If checkpointing is on
Checkpointing could revert files. It does not update the conversation.
- D

 Run /context first
Reports which memory files loaded. Nothing to do with filesystem state.

THE SIGNAL

Sessions persist the conversation; checkpoints persist the files. Module 4.

Scoring exam 3

- ◆ **Sixty questions answered.** Add this tally to the previous two before drawing any conclusion.
- ◆ **A domain weak in all three papers is your study plan.** Not the slides — the scenarios in those modules.
- ◆ **A domain that improved and held** is finished. Stop revising it.
- ◆ **Scores climbing while misses move around** means you are learning the material, not the papers. That is the good outcome.

ON THE EXAM

Three papers is a trend. Exam 4 is the check, not the diagnosis.

Which domain did you miss?

QUESTION NUMBERS	DOMAIN	MODULES TO REWATCH
1, 5, 8, 11, 15, 20	D1 Agentic architecture	1 to 4
6, 12, 16	D2 Tool design and MCP	5 to 7
2, 7, 13, 18	D3 Claude Code	8 to 11
3, 9, 14, 19	D4 Prompt and structured output	12 to 14
4, 10, 17	D5 Context and reliability	15 to 17

ON THE EXAM

Three tallies. If one row is weak every time, work that domain's scenarios before exam 4.